Loop Engineering

Operating AI coding agents that finish the job — and how it differs from harness engineering

A management briefing | Prepared by Duy Tran Thanh | duy.tran10@onemount.com

Executive summary

- **The shift.** In agentic coding, value has moved from *writing prompts* to *designing the loop* that runs an agent toward a goal. The leverage is no longer the single instruction; it is the system that generates and checks instructions until the work is done.
- **The distinction.** *Harness engineering* makes a single step reliable; *loop engineering* makes a whole goal *finish*, verified and within a budget. They are complementary, not competing: the loop is one component of the harness.
- **The payoff.** A loop converts a goal into an outcome that is *trustworthy* (only checked work counts), *auditable* (a full per-step trail), *cost-bounded* (a hard stop), and *resumable* (survives interruption). Capable agents run unattended without running away.
- **The mandate.** Apply loops where a *human is the bottleneck* on a repetitive, checkable, multi-step goal — not where output is merely unreliable (that is a harness problem). Insist on two non-negotiables: a built-in check and a hard stop.
- **How to start.** Express any task as six plain answers, run a reference loop in minutes, then scale into CI. A 90-day path is on the last page.

1. The bottleneck has moved from model capability to how we operate the model

Situation. Coding agents are now capable enough to do real, multi-step engineering work. **Complication.** Most teams still operate them by hand — prompt, read the result, re-prompt — which caps throughput at human attention and is error-prone at scale. **Question.** How do we drive a capable agent to a finished, trustworthy result *without* a human in every step?

The answer is to stop hand-steering and instead engineer the *loop*: a self-running cycle that pursues a goal, checks its own work, remembers progress, and stops on a defined condition.

Bottom line. The constraint is no longer “can the model do it?” but “how do we run it to a dependable finish?” That is an operating-system question, not a prompting question.

2. Harness and loop engineering solve two different problems — you need both

Think of the agent as *model plus harness*, where the harness is everything around the model. Two disciplines act on it (Exhibit 1):

- **Harness engineering** designs the scaffolding for one step — context, tools, guardrails, and checks — so that a single action is reliable. It asks: *“is this step good?”*
- **Loop engineering** designs the cycle that drives the agent across many steps toward a goal, and owns when to stop. It asks: *“is the whole goal met, and when do we stop?”*

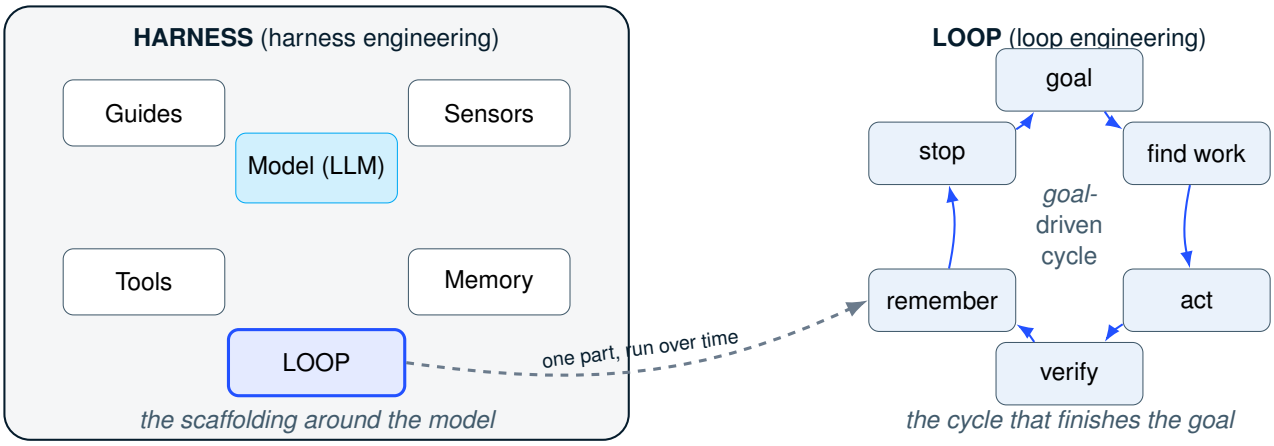

Exhibit 1. Loop engineering is the part of harness engineering you run over time. Harness engineering builds everything around the model; the loop is the cycle that drives it to a finish.

Dimension	Harness engineering	Loop engineering
What it designs	The scaffolding around the model: context, tools, guardrails, checks	The cycle that drives the agent to a goal
Scope	A single step	A whole run (many steps)
Core question	"Is this step good?"	"Is the goal met, and when do we stop?"
What it owns	Reliability of each action	Termination and verified completion
Input → output	Model + context → one checked action	Goal + means → a verified, auditable, resumable outcome
When to invest	Output is unreliable / the agent does the wrong thing	You are the bottleneck on a repetitive, checkable goal

Exhibit 2. The two disciplines are orthogonal: one improves a step, the other finishes a job. A mature operation invests in both.

The relationship is precise: the loop is *one component of the harness*, run over time. Harness engineering draws the box; loop engineering animates one part of it. Exhibit 2 contrasts them.

Bottom line. If the symptom is "wrong output," fix the harness. If the symptom is "the agent is good but I am stuck driving it," build a loop.

3. A loop converts a goal into a verified, auditable, bounded outcome

A loop is best understood by what goes in and what comes out (Exhibit 3).

- **Input — a goal plus the means to pursue and check it:** the objective (with a success test), the model, the workspace, any saved progress, and a verification/termination policy.
- **Output — a terminated, audited outcome:** *why* it stopped; the completed work, counting *only* items that passed their checks; the changed files; a replayable per-step trail; and a checkpoint that resumes the next run.

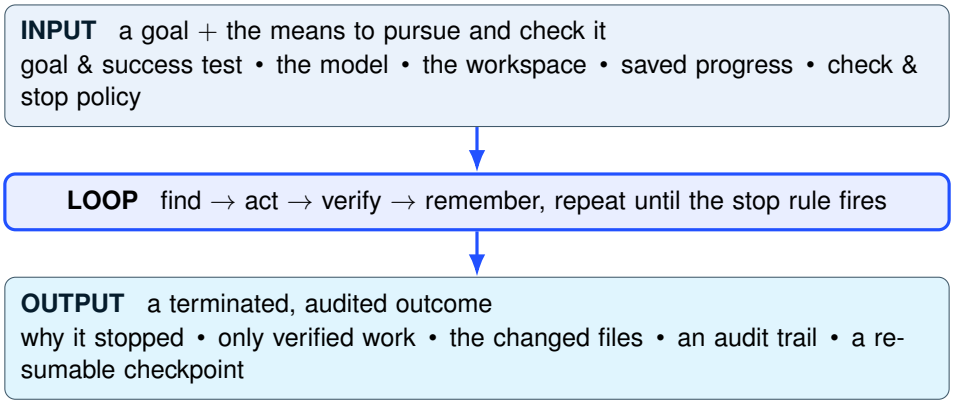

Exhibit 3. What a loop takes in and gives back. The output is trustworthy, accountable, cost-bounded, and resilient by construction.

For a decision-maker, the four output properties are the business case:

- **Trust** — only verified work is counted as done.
- **Accountability** — every step leaves an audit trail.
- **Cost control** — a hard budget and stop cap the spend.
- **Resilience** — an interrupted run resumes; nothing is redone.

Bottom line. A loop is not a faster prompt; it is a managed process that produces a checkable, accountable result with a known cost ceiling.

4. Apply loops where you are the bottleneck — not where output is simply unreliable

The investment decision is a simple triage:

- Output is unreliable, even on a single step → invest in the **harness** first (better checks, guides, tools).
- The agent is reliable per step but the goal is a repetitive, checkable, multi-step grind that you keep babysitting → invest in a **loop**.

Strong first candidates share three traits: a clear "done" test, many similar pieces of work, and low blast-radius per step. Exhibit 4 lists common ones.

Bottom line. Loops pay off fastest on repetitive, checkable work where a human is currently the rate limiter.

5. Implementation: six questions, one spec, runs anywhere

Turning a task into a loop means answering six plain questions and writing them in a short spec file:

1. **When is it done?** A command that succeeds or a check that comes back clean.
2. **What are the pieces of work?** How to list what still needs doing.
3. **How do you do one piece?** Usually: ask the model to fix one thing.
4. **How do you check it worked?** A test or linter, *wired before the actor.*
5. **How is progress remembered?** Saved to disk so a restart resumes.
6. **When should it stop?** Goal met, N tries, or a budget — always set one.

Questions 1, 4, and 6 are the non-negotiables: a checkable goal, a check before acting, and a hard stop.

Your task	Done when	One piece of work	How to check it
Make tests pass	the test suite passes	a failing test	re-run that test
Fix lint / format	the linter reports zero issues	a file with warnings	re-run the linter on it
Close all TODOs	no TODO comments remain	one TODO	TODO gone, tests still pass
Migrate config files	every file is in the new format	one old file	validate against the new schema
Upgrade a deprecated API	no calls to the old API remain	one call site	it still compiles and tests pass

Exhibit 4. Most jobs fit the same shape; you change only three answers — what "done" means, what one piece of work is, and how to check it.

The reference framework refuses to run a task that lacks a check or a stop.

The same loop runs in four places

By changing only *when* it runs and its limit, one loop becomes: a **nightly job** (run until nothing new is found); a **pull-request gate** (run once; block the merge unless done); a **bulk migration** (one file per piece, in isolated copies so parallel work does not clash); or a **supervised run** (pause for human approval every few steps).

Bottom line. The task definition does not change across these surfaces — only the trigger and the limit do. Build once, deploy four ways.

6. What to watch for

Four failure modes account for most trouble; each has a cheap mitigation:

- **No way to check "done."** The loop cannot tell success from failure. *Mitigation:* require an explicit check before building the loop.
- **Acting before checking.** The agent makes fast, unverified changes. *Mitigation:* wire the verifier first; count progress only when it passes.
- **Unstable work identifiers.** Work repeats after a restart. *Mitigation:* give each piece a stable name in saved state.
- **No stop limit.** The loop runs — and spends — forever. *Mitigation:* a mandatory budget and iteration cap.

Governance scales the same way: keep a human on the loop at defined checkpoints, set token and time budgets, and isolate parallel runs.

Bottom line. The two failures that cost the most — no check, no stop — are exactly the two the framework blocks before a loop can start.

7. Recommended next steps: a 90-day adoption path

1. **Weeks 1–2 — Prove the check.** Pick one repetitive, checkable task. Stand up its automated check (tests or linter). No loop yet.
2. **Weeks 3–6 — Run supervised.** Wrap the task in a loop with a hard stop. Run with a human on the loop. Measure hours saved and "escape rate" (bad changes that slipped a check).
3. **Weeks 7–12 — Scale into CI.** Promote the loop to a pull-request gate or nightly job. Add a second task. Set budgets, isolation, and governance checkpoints.

The one thing to remember

Treat the loop as a **managed system** with a goal, a check, and a stop — not as a clever prompt. That is what turns a capable agent into dependable, unattended throughput.

A companion technical report (`REPORT.md` / `loop-engineering.pdf`) provides the formal model, properties, and an open-source reference implementation (OnLOOP).